



# Ejercicios Propuestos: Memoria y Archivos

Malak Sanchez

Workshop 2026-II

16 de agosto de 2026

## Introduccion

El siguiente taller tiene como objetivo reforzar los conceptos fundamentales vistos durante el workshop: paso de parametros por linea de comandos, gestion de memoria dinamica y manipulacion de archivos. Los problemas estan organizados en tres niveles de dificultad progresiva para que pueda avanzar a su propio ritmo.

## Problema 1: Generador de Patron en Cruz

Escriba un programa que reciba por linea de comandos un numero entero impar  $N$ , cree dinamicamente una matriz bidimensional de caracteres de  $N \times N$  y dibuje en ella un patron en forma de 'X' usando asteriscos ('\*') y espacios (' '). Una vez construida, el programa debe imprimirla.

**Ejemplo de ejecucion:**

```
$ ./pattern 5
```

**Ejemplo de salida:**

```
*   *  
*  *  
 *  
*  *  
*   *
```

**Requerimientos:**

1. Valide que se reciba exactamente un argumento adicional y que sea un numero impar positivo.
2. Reserve memoria dinamica para una matriz bidimensional.
3. Llene la matriz con el patron sin imprimir directamente durante la asignacion. Use un bucle anidado.
4. Imprima la matriz resultante en pantalla usando un recorrido fila por fila.
5. Libere correctamente toda la memoria reservada antes de terminar el programa.

## Problema 2: Buscador de Palabras en Archivo

Disene un programa que busque una palabra especifica dentro de un archivo de texto. El programa debe imprimir la cantidad total de veces que aparece la palabra en el archivo y mostrar cada linea completa donde hubo una coincidencia.

**Ejemplo de comando:**

```
$ ./searcher log.txt "error"
```

**Ejemplo de salida esperada:**

```
Coincidencia en linea 3: FATAL error in connection.  
Coincidencia en linea 8: timeout error occurred.  
Total de apariciones: 2
```

**Requerimientos:**

1. Valide que se pasen exactamente dos argumentos (el nombre del archivo y la palabra clave).
2. Valide que el archivo pueda abrirse correctamente, manejando cualquier posible error.
3. Reserve memoria dinamica para un buffer temporal de lectura.
4. Lea el archivo linea por linea empleando la funcion `fgets`. Para buscar la coincidencia dentro de la linea puede apoyarse en la funcion `strstr`.
5. Imprima las lineas que coincidan junto con su numero de linea correspondiente y mantenga un contador del total de ocurrencias.
6. Libere toda la memoria dinamica usada y asegurese de cerrar el archivo.

## Problema 3: Cifrado y Redimensionamiento Dinamico

Implemente un programa que reciba un archivo de texto de entrada y uno de salida. El programa debe leer todo el contenido del archivo fuente en un unico bloque de memoria que comienza pequeño pero crece dinamicamente segun sea necesario. Finalmente, debe aplicar un cifrado simple (por ejemplo, sumar 3 al valor ASCII de cada caracter) y guardarlo en el archivo de destino.

### Ejemplo de comando:

```
$ ./cipher input.txt output.txt
```

### Ejemplo de salida:

```
Archivo input.txt leido exitosamente.  
Contenido cifrado y guardado en 'output.txt'.
```

### Requerimientos:

1. Valide que se pasen exactamente dos argumentos en linea de comandos (archivo de entrada y archivo de salida).
2. Intente abrir ambos archivos con los modos adecuados y valide que los punteros devueltos no sean NULL.
3. Implemente una funcion `encrypt(char *buffer, int size)` que recorra la memoria y aplique un desplazamiento a cada caracter.
4. Escriba el bloque de memoria procesado completo en el archivo de salida usando `fwrite`.

## Criterios de Evaluacion

- **Gestion de Memoria:** Uso correcto de `malloc` y `free`; ausencia de fugas de memoria.
- **Robustez:** El programa no debe fallar si el archivo no existe, si los argumentos son insuficientes o si los datos son invalidos.
- **Claridad:** Codigo bien estructurado, con funciones separadas y comentarios en los puntos clave.